PCCPU

James Augustus

In one sentence

PCCPU is a separate controller unit that does no calculation and hands out the work. It is a design I am still working on. It is not finished, and I have not placed it in the single core version of Hunter.

The idea

My older CT-CPU design had 4096 cores. I added a 4097th core to them and called it PCCPU (Program Counter CPU). It does not calculate, it blows the whistle. It sets the rhythm, which is the clock. It keeps the order, which is the counter. It says who gets what, which is the address. It does not have to know what is inside an instruction, it only hands it out. It is like a traffic officer or a card dealer.

I corrected a mistake in the design. At first I thought PCCPU fed an instruction on every clock tick. Feeding four thousand cores one by one would be a bottleneck. What is right is this: PCCPU starts the work and steps back, the cores work independently, and each one has its own counter and its own loop. PCCPU is only called back when something concerns the outside: when there is an overflow, when data has to pass between cores, when a job is finished and when synchronisation is needed. The whistle blower says “start”, and the athletes do their own training. The whistle blows only for a collision, for time or for a change, it does not call out every movement.

Three ways of working

In the central way, PCCPU sends every instruction one by one and the cores do what they are told. It is sequential, with no parallelism. In the independent way, PCCPU hands out the work and watches, and each core runs its own program. In the cube and leader way, the cores are split into cubes, PCCPU talks only to the leaders, and the worker cores only calculate.

I think choosing one way is wrong, the three should work together. When the system starts I load the data in the central way, then I build the cubes, the idle cores work independently, and PCCPU decides which one is active and when.

What is still open

Where the program is kept is open: in the registers, in a small instruction buffer in every core, or broadcast from PCCPU. Where PCCPU itself sits is open too: outside the grid, inside it, or in software. What goes inside it is still under discussion: a program counter, an address decoder, an instruction decoder, and maybe a table that records whether each core is busy or idle. Whether more than one PCCPU is needed I will look at later, starting with one and checking again if a bottleneck appears.

Hunter is a single core today. There, the place of the instruction source and the program counter is not decided yet. I do not put the old PCCPU into that gap as a finished part. This document describes where the design stands now, it does not show a tested circuit.

Address-based execution model

I prepared a small C model in which values, tasks and controller state share an address scheme. Workers at 1000 and 1001 produce C(9,3) = 84 and C(9,4) = 126. The worker at 1002 waits for both and writes their sum to 1003. The result is C(10,4) = 210. Controller state is at 1004.

These addresses are record identifiers, not physical RAM byte addresses. Worker routines perform arithmetic. The controller tracks readiness and completion. The example's completion counter is neither a program counter nor a timing measurement. Three worker records do not mean three physical cores. This C model executes sequentially.

The repository includes the C source, runner and a step-by-step Turkish explanation.

Mathematical basis: Pascal rows and N(p)

Pascal coefficients satisfy C(n,k) = C(n,n-k). For every n ≥ 0,

`(x+1)^n = Σ C(n,k) x^k, k = 0,...,n.`

This supports associating equal coefficients with two different exponent positions. Equal coefficients do not imply equal weighted terms. C(n,k) x^k and C(n,k) x^(n-k) generally have different values.

To read the coefficients directly as digits, choose an integer base x = b ≥ 2. Let c(n) = C(n,⌊n/2⌋), the largest coefficient. Direct digit interpretation requires c(n) < b.

For a Mersenne base b = 2^p - 1 with integer p ≥ 2, define

`N(p) = max { n ≥ 0 : C(n,⌊n/2⌋) < 2^p - 1 }.`

Carry-free rows are precisely 0 through N(p). Thus n ≤ N(p) permits a symmetric coefficient record to be used directly as a symmetric digit record. N(p) bounds that mathematical interpretation. It does not determine scheduling, RAM layout or hardware performance.

For p = 3, b = 7 and N(3) = 4. Row 4 has central coefficient 6 < 7, whereas row 5 has 10 ≥ 7. The coefficients [1,4,6,4,1] of row 4 are the base-7 digits of 4096. Row 5 still has symmetric coefficients, but carrying produces digits [1,6,4,3,5,1]. Their digit symmetry cannot be assumed.

The integer sum 84 + 126 = 210 remains valid. At b = 127, however, N(7) = 9, so the resulting row n = 10 is no longer carry-free. Choosing p = 8 and b = 255 gives central coefficient 252 < 255 and N(8) = 10. Arithmetic correctness and carry-free representation in a selected base are separate checks.

This section builds on the positional representation in my Pascal project. The published mathematical note describes the selected row and carry-free condition. N(p), defined here, identifies the last row satisfying that condition at a fixed base. Symmetry comes from Pascal coefficients. N(p) limits when they can be read as digits.

One thread owns two symmetric positions

I propose that one thread record own two symmetric positions in the same row. Here a thread is a logical task identity, not necessarily an operating-system thread or a physical core. For degree n = 4:

Record	Shared coefficient	First position	Mirrored position
thread_no_1	1	x^0	x^4
thread_no_2	4	x^1	x^3
thread_no_3	6	x^2	The same central position

thread_no_1 may store the values of x^0 and x^4 in separate fields. The first is 1 and the second is x^4. Their shared coefficient is 1, which need not be stored twice. The two power values are not assumed equal.

At x = 7, thread_no_1 has power values 1 and 2401. thread_no_2 has powers 7 and 343, with weighted contributions 28 and 1372. The central contribution is 6 × 49 = 294. The total is 1 + 2401 + 28 + 1372 + 294 = 4096, counting the centre once.

A possible notation is `T(n,k)[a]`. The index k denotes an exponent position counted from the right, with $0 \le k \le \lfloor n/2 \rfloor$. The selector a is either 0 or 1. It selects exponent k or n-k, respectively. Thus `T(4,0)[0]` selects x^0 on the right, and `T(4,0)[1]` selects x^4 on the left. This is proposed notation, not a redefinition of N(p) or a final specification of the earlier multi-argument N notation.

C(4,1) = C(4,3) = 4 means that both positions have coefficient 4. The operations here are not 4^1 and 4^3. A shared coefficient 4 multiplies x^1 and x^3. Raising the coefficient itself to a positional power would require a distinct operation.

A RAM record preserving symmetry

A proposed record stores n, k, x or a reference to x, and a shared coefficient. Storing both expanded powers requires two value fields with separate readiness information. A thread identity indicates ownership, while an address locates the record.

Field	Meaning
n, k	Row and the first exponent position of the pair
x or x_ref	Power base or reference to it
coefficient	Shared C(n,k)
power[0], power[1]	Values x^k and x^(n-k), when ready
ready[0], ready[1]	Separate readiness for each side
owner	Logical thread owning the record

Storing only n, k, x and the coefficient is another option. Then the values are computed on demand. Storing both complete powers costs computation and memory. These are different representations, and neither is fixed as a hardware decision here.

At n = 4, k = 2, both selectors refer to the same x^2 position. It can be stored once and must be added once. At n = 3, pairs are (0,3) and (1,2). Equal central coefficients still occupy two distinct positions, and both contributions count.

Changing x, n or k invalidates previously computed powers. Independently changing one coefficient must also break the shared-coefficient relationship. Otherwise the model would impose a false symmetry. Write, validity and completion rules belong in the RAM protocol.

Only $\lfloor n/2 \rfloor + 1$ distinct coefficients are needed for a symmetric Pascal row. Total hardware memory need not decrease by the same factor. Expanded values, addresses and state bits occupy additional space. Ordinary RAM can store this record layout. Special-purpose RAM hardware still needs separate design and testing.

Implementation status and execution

The C example implements two producers and an adder with readiness dependencies. It compares 20,100 adjacent Pascal sums with an addition-only construction. Including boundary and state checks, it evaluates 60,353 conditions. From the repository root:

```
bash kod/PCCPU/calistir.sh
```

A C11 compiler and GMP development files are required. Symmetric thread records, paired RAM values and side selection are not implemented in this C program yet. The added sections specify their mathematical meaning and proposed record contract. The current program’s test count is not validation of that new RAM design.

Text: CC BY 4.0. C example: MIT license.